

Deep Learning Techniques for Image-based Thai Food Classification

Chanon Aupattanapanit*

Department of Statistics, Chulalongkorn University

Abstract

This article studies the classification of Thai food from images and constructs and compares models for the task using convolutional neural networks (CNNs). The data are Thai food images from the Kaggle website; the volume of data was reduced because of resource constraints on model training. The resulting dataset covers 12 food categories, each split into three subsets: 240 images for training, 30 for validation, and 30 for testing, or 3,600 images in total. We apply transfer learning to build and compare CNN models over four architectures — (1) VGG16, (2) ResNet50, (3) InceptionV3, and (4) Xception — under two training schemes: (1) feature extraction and (2) fine-tuning of the weights and parameters in the model. The fine-tuned Xception architecture performs best among all architectures, with an accuracy of 93.06%, precision of 93.22%, recall of 93.06%, and F1-score of 93.02%.

Keywords: Thai food, convolutional neural network, transfer learning

1 Introduction

Since the surge of tourism in Thailand in the 1980s, Thai food has established itself as one of the world’s leading culinary traditions and ranks among the five most popular cuisines in the world, alongside Indian, Chinese, French, and Italian cuisine (Khaolakexplorer, n.d.). Traditional Thai food falls into four categories — *tom* (boiled dishes), *yam* (spicy salads), *tam* (pounded dishes), and *gaeng* (curries) — while frying, stir-frying, and steaming were adopted from Chinese cuisine [21].

According to Mastercard’s Global Destination Cities Index [22], Bangkok, the capital of Thailand, has been the most visited city in the world since 2016, with more than 38 million visitors from around the globe, and most visitors to Thailand each year are foreigners. Communicating with local people, above all when ordering food, can therefore be difficult and confusing, because Thai food is highly diverse and dishes carry different names even when they look alike (Gary, 2018).

Artificial intelligence (AI) is today the discipline at the heart of giving technology human-like intelligence and learning ability, a capability known as machine learning (ML). Deep learning is one such method: it designs computer networks that imitate the working of the human brain by stacking artificial neural networks in many layers and learning from sample data (Vithan, 2018). The learned representations are then used to detect patterns or to group data, and the convolutional neural network (CNN) is one of the algorithms that supports the processing of image data [1].

For these reasons, we set out to study the classification of Thai food from images with convolutional neural network models and to compare CNN models for the task using transfer learning, so that the resulting models can assist in identifying Thai food.

*Statistics Project SP65-11. Email: chanon.aupattanapanit@gmail.com

1.1 Objectives

1. To study the classification of Thai food from images using convolutional neural network models.
2. To construct and compare convolutional neural network models for classifying Thai food from images using transfer learning.

1.2 Scope of the study

This article uses Thai food images in JPEG format from the Kaggle website, comprising 12 categories of Thai food with 300 images per category, for a total of 3,600 images.

The remainder of this article is organised as follows. Section 2 reviews the theoretical background — convolutional neural networks and transfer learning, including the four pre-trained architectures used here — and surveys related work. Section 3 describes the research tools, the data, and the construction of the models. Section 4 reports the experimental results, Section 5 concludes and discusses the findings, and Section 6 offers recommendations for future work. Appendix A defines the evaluation metrics, and Appendices B and C provide background on data augmentation and on the activation functions used.

2 Literature Review

This section first reviews the concepts and theories on which the study rests — convolutional neural networks (Section 2.1) and transfer learning with the four pre-trained architectures (Section 2.2) — and then surveys related work on food image classification (Section 2.3).

2.1 Convolutional neural networks

A convolutional neural network (CNN) [1, 2] is an artificial neural network that models human vision. The idea grew out of studies of the human visual cortex, and CNNs have been used for image recognition since the 1980s [24]. As computing power has grown, CNNs have become able to digest enormous amounts of data, so that trained models now surpass humans on certain complex visual tasks such as image search, self-driving cars, and automatic video classification. Nor are CNNs limited to visual perception: they have also succeeded in many other tasks, such as speech recognition and natural language processing (NLP). Figure 1 shows the structure of a CNN.

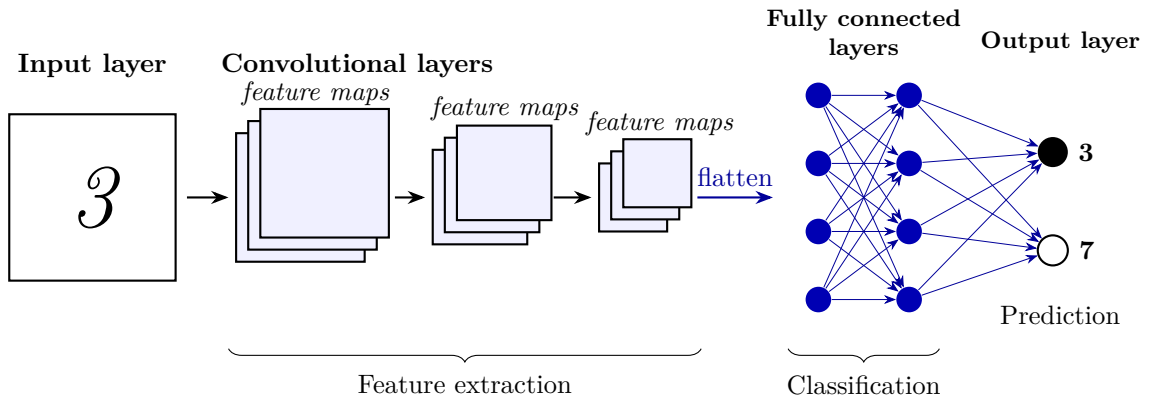


Figure 1. Architecture of a convolutional neural network: convolutional layers extract feature maps from the input image, and fully connected layers classify the flattened features (redrawn after Géron [1]).

The basic structural components of a convolutional neural network are as follows.

2.1.1 Convolutional layer (CONV)

The convolutional layer is the core component of a CNN. It extracts features from the image, such as edges and colour contrasts. Mathematically, the convolution operation combines two functions to produce a third, modified function; in the CNN setting, the first function is the input image and the second is the filter (CONV filter). The filter slides over the input image, splitting it into small patches and processing them one at a time to assemble the output image.

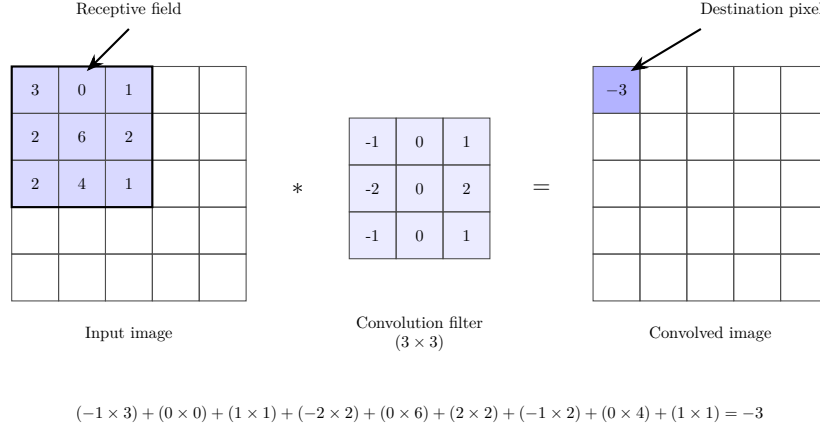


Figure 2. The convolution operation in a CNN: a 3×3 filter slides over the input image, and the weighted sum over each receptive field produces one pixel of the convolved image (redrawn after Elgendy [2]).

As Figure 2 illustrates, a small 3×3 matrix called a *kernel* slides over the original image pixel by pixel and computes a weighted sum to obtain the value of the new image in the next layer, an operation known as *convolving*; the region of the image that has been convolved is called a *feature map* [4].

The kernel size (filter size) is one of the hyperparameters fixed when a CNN architecture is designed. A smaller kernel captures image detail more finely, whereas a larger kernel separates detail more coarsely. Kernels are usually square, ranging from 2×2 (the smallest) to 5×5 (the largest). In principle a model can use still larger kernels, but doing so is inadvisable because important image detail is lost (Figure 3).

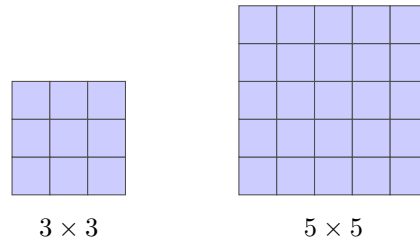


Figure 3. Kernel sizes of 3×3 and 5×5 (redrawn after Elgendy [2]).

Padding. Padding [3] extends the borders of the input equally on every side in order to preserve the spatial dimensions of the image; for instance, a 6×6 input padded with $p = 1$ becomes 8×8 . Most often zeros are added around the edge of the image, which is called *zero-padding* (Figure 4). Padding keeps the width and height of the input and output images equal, which matters when building deep CNN architectures, because the convolution operation shrinks the width and height of the image as it passes through deeper layers. With padding, the output image size is

$$(n + 2p - f + 1) \times (n + 2p - f + 1), \quad (1)$$

where

n is the dimension of the input image,
 p is the amount of border extension, and
 f is the kernel size.

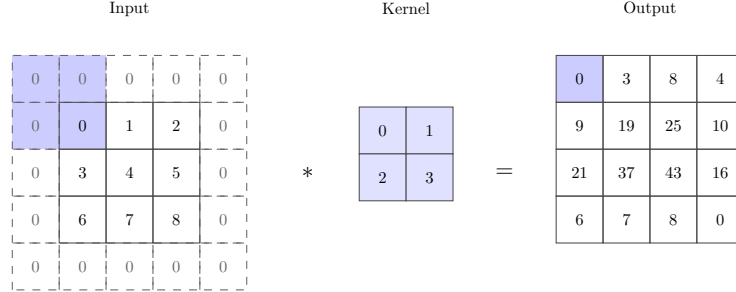


Figure 4. Zero-padding: adding zeros around the border of the image preserves its spatial size after convolution (redrawn after Dive into Deep Learning [3]).

Stride. The stride [3] is the distance the kernel moves across the image. Setting a larger stride shrinks the feature-map matrix, which is one way of countering *overfitting*, where the model memorises the training data (Figure 5). With a stride, the output image size is

$$\left\lfloor \frac{n + 2p - f}{s} \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} \right\rfloor, \quad (2)$$

where

n is the dimension of the input image,
 p is the amount of border extension,
 f is the kernel size, and
 s is the distance the kernel moves across the image.

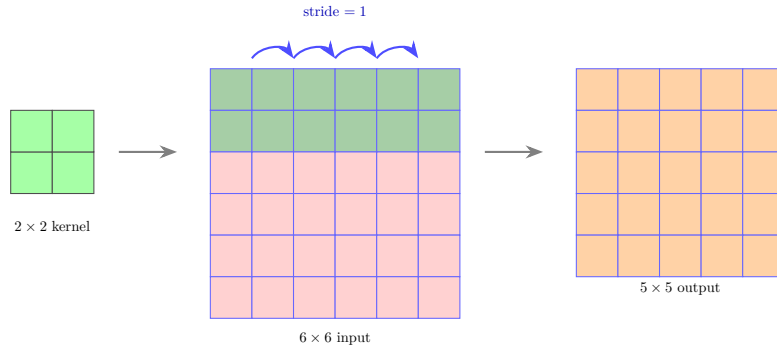


Figure 5. Stride: a 2×2 kernel moving one pixel at a time (stride = 1) over a 6×6 input produces a 5×5 output (redrawn after MYO NeuralNet [4]).

2.1.2 Pooling layer (POOL)

The pooling layer subsamples the feature maps created in the convolutional layer so that they carry fewer parameters, which reduces the computational load. A POOL layer is typically inserted after every one or two convolutional layers in a CNN architecture. Two mechanisms are common:

1. **Max pooling** extracts features from the input image by keeping only the maximum value within each kernel and storing it in the output image (Figure 6).
2. **Global average pooling** extracts features by averaging the values within the kernel and storing the mean in the output image (Figure 7).

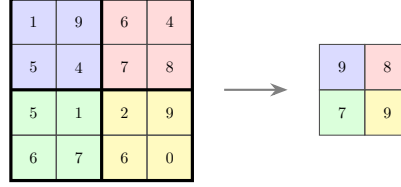


Figure 6. Max pooling with a 2×2 kernel: each shaded window of the input keeps only its maximum value in the output (redrawn after Elgendy [2]).

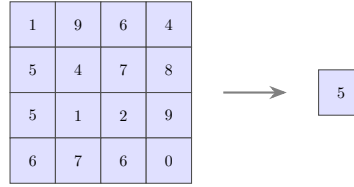


Figure 7. Global average pooling: the output is the mean of the values in the input feature map (redrawn after Elgendy [2]).

2.1.3 Fully connected layer (FC)

The fully connected layer [5] has the following structure (Figure 8):

1. **Input layer:** receives the result of the pooling layer with the feature-map matrix flattened into a single long vector; the number of nodes depends on the dimensions of the image.
2. **Hidden layer:** receives data from the preceding layer and processes it. Each time the model learns from the image vector, the nodes in the hidden layer gradually adjust their weights to fit the data; there can be more than one hidden layer.
3. **Output layer:** receives the values of the last hidden layer and produces the result; the number of nodes in this layer depends on the classes in the dataset.

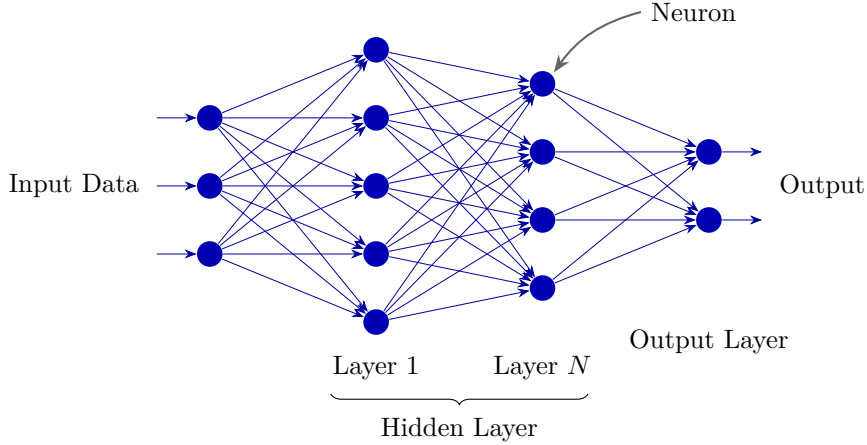


Figure 8. A fully connected neural network: input, hidden, and output layers of neurons (redrawn after Moolayil [5]).

2.2 Transfer learning

Transfer learning [6, 31] focuses on storing the knowledge gained while solving one problem and applying it to a different one. It shortens model training by reusing part of an architecture that has already been trained on a similar task with a large amount of data, so that its weights are already well adjusted, as a component of a new model, either to extract features for the target task or to work with specialised data of which only a little is available. We applied transfer learning with four architectures: (1) VGG16, (2) ResNet50, (3) InceptionV3, and (4) Xception, all of them popular choices for training image-classification models. Each architecture is described below.

2.2.1 VGG16

VGG16 [7, 25], also known as VGGNet, is a 16-layer-deep convolutional neural network devised at the University of Oxford. The architecture is built around CONV2D layers with 3×3 -pixel filters, a stride of 1, and zero-padding, together with 2×2 -pixel max-pooling layers with a stride of 2, repeated in the same pattern throughout (a uniform architecture); see Figure 9 and Table 1. With this design, VGG16 reached a top-5 accuracy of 90.1% on ImageNet — a dataset of more than 14 million training images in 1,000 object classes — in the 2014 ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [26].

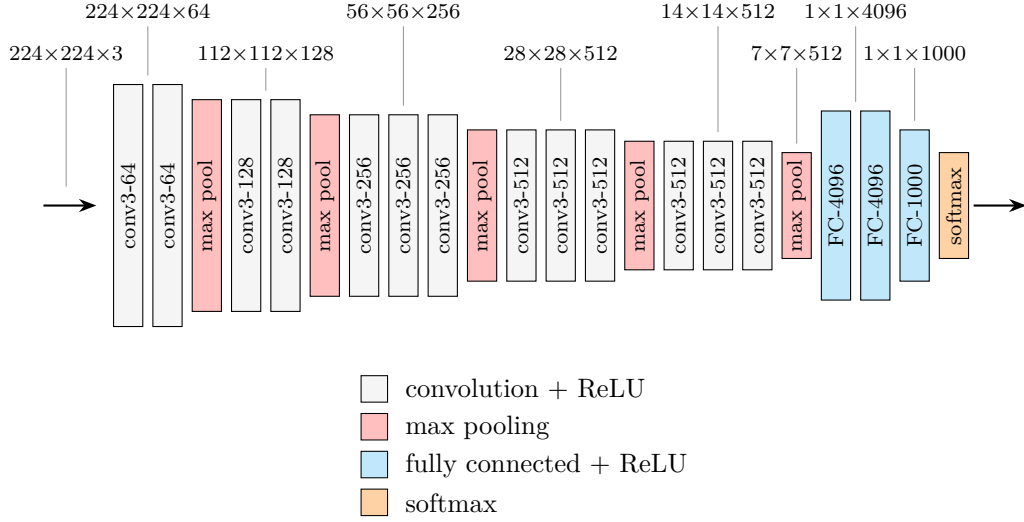


Figure 9. The VGG16 architecture: thirteen 3×3 convolutional layers interleaved with 2×2 max-pooling layers, followed by three fully connected layers and a softmax output (redrawn after Datagen [7]).

Table 1. Structural details of VGG16: layer-by-layer input and output sizes, kernels, strides, and parameter counts (after Datagen [7]; parameter counts follow the Keras implementation of VGG16).

#	Input size	Output size	Layer	Stride	Kernel	In	Out	Parameters
1	$224 \times 224 \times 3$	$224 \times 224 \times 64$	conv3-64	1	3×3	3	64	1,792
2	$224 \times 224 \times 64$	$224 \times 224 \times 64$	conv3-64	1	3×3	64	64	36,928
	$224 \times 224 \times 64$	$112 \times 112 \times 64$	maxpool	2	2×2	64	64	0
3	$112 \times 112 \times 64$	$112 \times 112 \times 128$	conv3-128	1	3×3	64	128	73,856
4	$112 \times 112 \times 128$	$112 \times 112 \times 128$	conv3-128	1	3×3	128	128	147,584
	$112 \times 112 \times 128$	$56 \times 56 \times 128$	maxpool	2	2×2	128	128	0
5	$56 \times 56 \times 128$	$56 \times 56 \times 256$	conv3-256	1	3×3	128	256	295,168
6	$56 \times 56 \times 256$	$56 \times 56 \times 256$	conv3-256	1	3×3	256	256	590,080
7	$56 \times 56 \times 256$	$56 \times 56 \times 256$	conv3-256	1	3×3	256	256	590,080
	$56 \times 56 \times 256$	$28 \times 28 \times 256$	maxpool	2	2×2	256	256	0
8	$28 \times 28 \times 256$	$28 \times 28 \times 512$	conv3-512	1	3×3	256	512	1,180,160
9	$28 \times 28 \times 512$	$28 \times 28 \times 512$	conv3-512	1	3×3	512	512	2,359,808
10	$28 \times 28 \times 512$	$28 \times 28 \times 512$	conv3-512	1	3×3	512	512	2,359,808
	$28 \times 28 \times 512$	$14 \times 14 \times 512$	maxpool	2	2×2	512	512	0
11	$14 \times 14 \times 512$	$14 \times 14 \times 512$	conv3-512	1	3×3	512	512	2,359,808
12	$14 \times 14 \times 512$	$14 \times 14 \times 512$	conv3-512	1	3×3	512	512	2,359,808
13	$14 \times 14 \times 512$	$14 \times 14 \times 512$	conv3-512	1	3×3	512	512	2,359,808
	$14 \times 14 \times 512$	$7 \times 7 \times 512$	maxpool	2	2×2	512	512	0
14	$1 \times 1 \times 25088$	$1 \times 1 \times 4096$	fc	–	1×1	25088	4096	102,764,544
15	$1 \times 1 \times 4096$	$1 \times 1 \times 4096$	fc	–	1×1	4096	4096	16,781,312
16	$1 \times 1 \times 4096$	$1 \times 1 \times 1000$	fc	–	1×1	4096	1000	4,097,000
Total								138,357,544

2.2.2 InceptionV3

InceptionV3 [8, 28] is an architecture developed by Google that grew out of the Inception module in GoogLeNet [27]. It concentrates on making the network deeper and wider without increasing the number of parameters, so as to process more complex relationships efficiently. InceptionV3 reaches a top-5 accuracy of 93.7%, can classify up to 1,000 classes, and won the ILSVRC competition in 2015 (Figure 10).

The distinctive feature of InceptionV3 is the Inception module, which reduces the number of parameters and matrix multiplications in the convolution layer by using 1×1 convolution blocks, known as *bottleneck* layers, so that each layer of the model trains faster. The data from the previous layer are fed into four different branches, as Figure 11 shows; Table 2 lists the layer

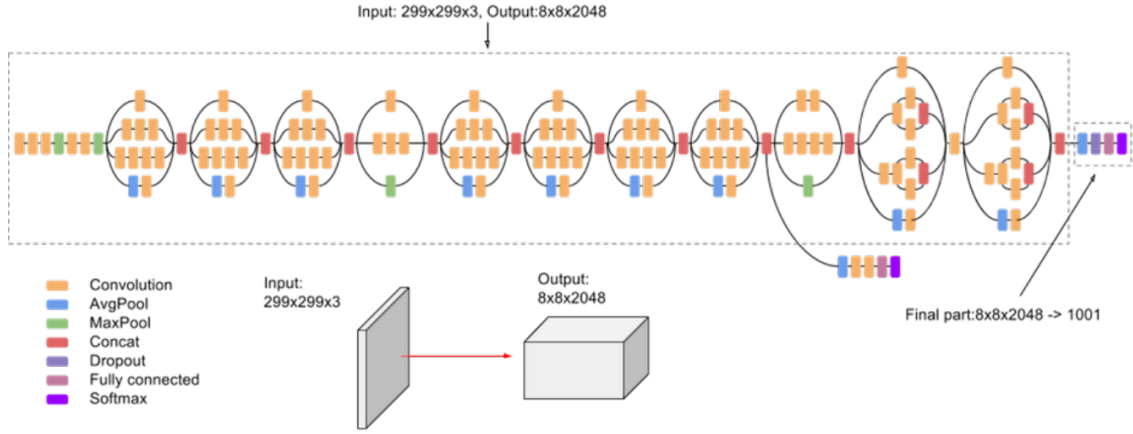


Figure 10. The InceptionV3 architecture (source: Google Cloud [23]).

specification.

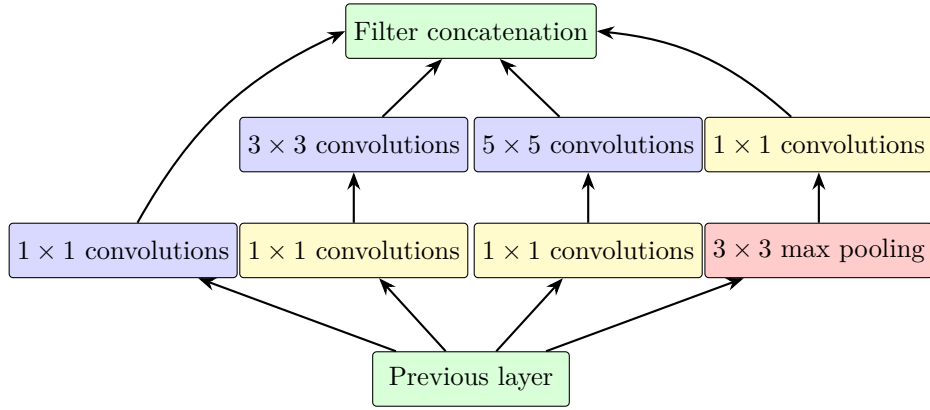


Figure 11. The Inception module: the previous layer feeds four parallel branches whose outputs are concatenated; 1×1 bottleneck blocks keep the parameter count down (redrawn after Sarkar [8]).

Table 2. Layer specification of the InceptionV3 network (after Szegedy et al. [28]); “figure 5”–“figure 7” in the remarks column refer to the Inception module variants in the original paper.

Type	Patch size/stride or remarks	Input size
conv	$3 \times 3/2$	$299 \times 299 \times 3$
conv	$3 \times 3/1$	$149 \times 149 \times 32$
conv padded	$3 \times 3/1$	$147 \times 147 \times 32$
pool	$3 \times 3/2$	$147 \times 147 \times 64$
conv	$3 \times 3/1$	$73 \times 73 \times 64$
conv	$3 \times 3/2$	$71 \times 71 \times 80$
conv	$3 \times 3/1$	$35 \times 35 \times 192$
3× Inception	As in figure 5	$35 \times 35 \times 288$
5× Inception	As in figure 6	$17 \times 17 \times 768$
2× Inception	As in figure 7	$8 \times 8 \times 1280$
pool	8×8	$8 \times 8 \times 2048$
linear	logits	$1 \times 1 \times 2048$
softmax	classifier	$1 \times 1 \times 1000$

2.2.3 ResNet50

ResNet [9, 10, 29], short for Residual Network, is an architecture that overcomes the vanishing-gradient problem by chaining many residual units together. ResNet50 denotes the variant that is 50 layers deep, consisting of 48 convolution layers, one max-pooling layer, and one average-pooling layer; see Figure 12 and the layer configuration in Table 3. ResNet50 achieved a top-5 accuracy of 92.1% in the ILSVRC competition in 2015.

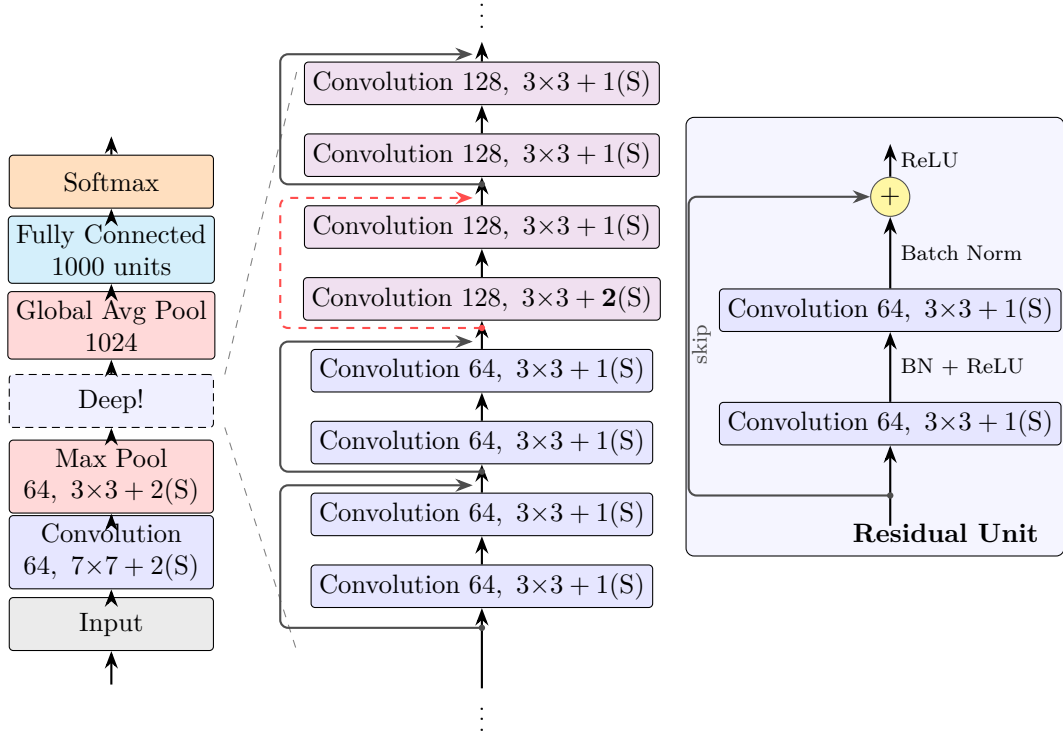


Figure 12. The ResNet50 architecture: the overall stack (left), the stacked residual layers inside it (centre; each skip line adds the input of a pair of convolutions to its output, and the dashed red line marks the skip across a stride-2 dimension change), and a residual unit with its skip connection feeding the addition point + (right) (redrawn after Adam [9]).

Inside the residual unit there is a shortcut, the *skip connection*, which raises the model’s

performance: if any layer in the model learns the data poorly compared with the others, the model skips it in favour of a deeper layer that learns better than its predecessor. Skip connections also remove the *degradation problem*, in which making the model deeper lowers its performance on both the training and the test data (Figure 13).

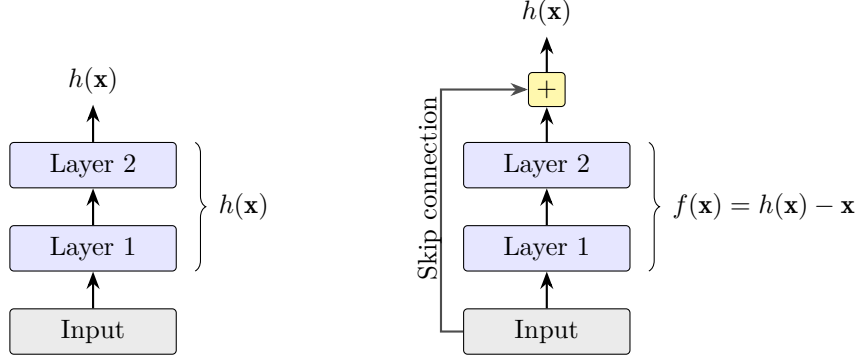


Figure 13. A plain stack of layers learns $h(\mathbf{x})$ directly (left); with a skip connection, the layers learn the residual $f(\mathbf{x}) = h(\mathbf{x}) - \mathbf{x}$ instead (right) (redrawn after Adam [9]).

Table 3. Layer configurations of the ResNet family; the 50-layer column corresponds to ResNet50 (after He et al. [29]). Building blocks are shown in brackets, with the numbers of blocks stacked; down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

Layer name	Output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	$7 \times 7, 64, \text{stride } 2$				
conv2_x	56×56	$3 \times 3 \text{ max pool, stride } 2$				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
1×1		average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

2.2.4 Xception

Xception [12, 30] is an architecture that refines the Inception module and extends the Inception line, while also adopting the skip-connection idea from ResNet. It is 71 layers deep and was created in 2016 by François Chollet, the author of the Keras library. Xception reaches a top-5 accuracy of 94.5% (Figure 14).

What sets this architecture apart is its separate detection of image characteristics in the convolution layer — the *depthwise separable convolutional layer* — which replaces the Inception module. It consists of two parts: (1) a spatial filter for each feature map, and (2) a filter for the patterns specific to each channel, implemented with 1×1 convolutional blocks. These blocks cut down the computation and make training more efficient (Figure 15).

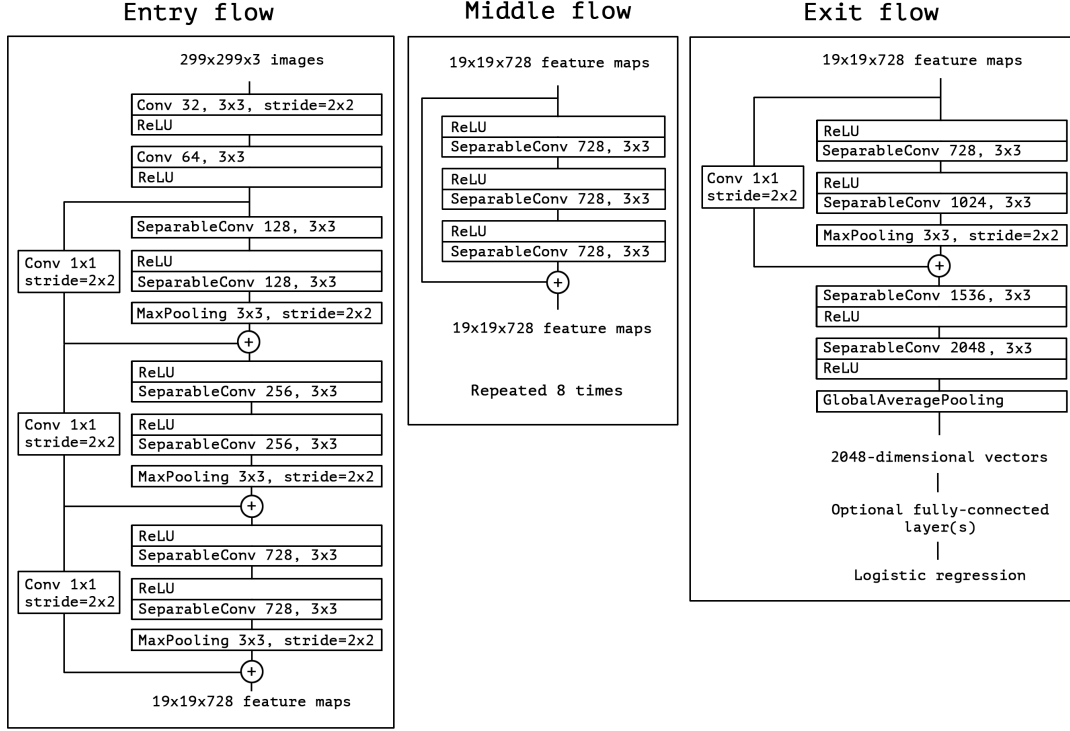


Figure 14. The Xception architecture: entry, middle, and exit flows (source: Chollet [30]).

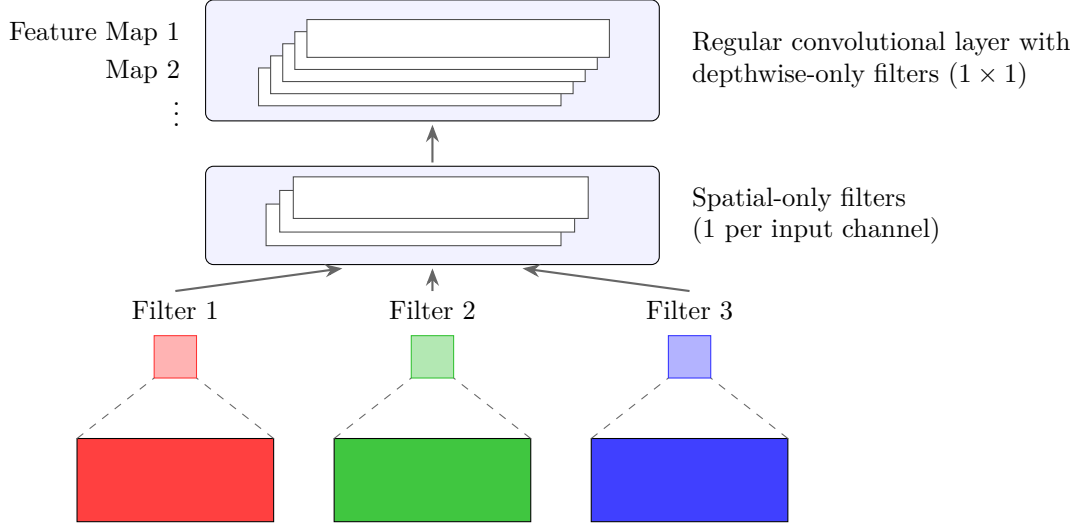


Figure 15. The depthwise separable convolutional layer: a spatial-only filter is applied to each input channel, and 1×1 depthwise-only filters then combine the patterns across channels (redrawn after Geron [1]).

2.3 Related work

Chayanat et al. (2022) [18] compared models for classifying 50 categories of Thai food with convolutional neural networks, using seven pre-trained architectures: MobileNetV1, MobileNetV2, MobileNetV3, EfficientNetV1, EfficientNetV2, ResNet50V1, and ResNet50V2. They compared performance under three training regimes — training from scratch, fine-tuning the last layer, and fine-tuning the whole network — and under four levels of data augmentation: 0%, 5%, 10%, and 15%. EfficientNetV1 with 10% data augmentation gave the best results of all the architectures, with an accuracy of 91.49%.

Vanuttre (2022) [19] compared models for classifying 101 food categories with convolutional

neural networks, using five pre-trained architectures: MobileNetV2, InceptionV3, EfficientNetB2, ResNet152, and ResNet50. InceptionV3 obtained the highest accuracy, 93.89%, followed by ResNet152.

Nguyen et al. (2021) [20] compared models for classifying 21 categories of Vietnamese food with convolutional neural networks, using six pre-trained architectures including VGG19, ResNet50, DenseNet121, Xception, and EfficientNetB0, with a support vector machine (SVM) as the classification layer. On the test set, the fine-tuned EfficientNetB0 gave the highest accuracy, 74.81%, followed by Xception.

3 Research Methodology

This section describes the research tools, the collection and preparation of the image data, and the construction of the classification models.

3.1 Research tools

1. The Python language on Google Colab, for analysing the image data and building the Thai food classification models.
2. Convolutional neural networks.

3.2 Data collection

This study uses Thai food images in JPEG format from the Kaggle website — secondary data scraped from various websites with Python. The collection consists of 48 categories of Thai food images, 300 images per category, for a total of 14,400 images.

3.3 Data preparation

3.3.1 Data exploration and splitting

Starting from the source data, we reduced the number of food categories to 12 because of resource constraints on model training; Table 4 lists the selected dishes.

Table 4. The food list: each dish’s class code with its Thai and English names.

Class code	Thai name	English name
00	แกงเขียวหวาน	Green Curry
01	ส้มตำ	Green Papaya Salad
02	ข้าวซอย	Khao Soi
03	ข้าวเหนียวมะม่วง	Mango Sticky Rice
04	แกงมัสมั่น	Mussaman Curry
05	ผัดกะเพรา	Pad Kra Pao
06	ผัดไทย	Pad Thai
07	ข้าวคลุกกะปิ	Rice mixed with Shrimp Paste
08	แกงส้ม	Sour Soup
09	แกงเลียง	Spicy mixed vegetable soup
10	แกงเทโพ	Tepho Curry
11	ต้มยำกุ้ง	Tom Yum Kung

We examined the class balance of the data and found that each of the 12 Thai food categories holds exactly 300 images. The data were then split into three parts — a train set, a validation set, and a test set — in proportions of 80%, 10%, and 10%, giving 240, 30, and 30 images per category, respectively. Figure 16 shows one training image from each of the 12 categories.



Figure 16. Sample training images for the 12 categories of Thai food.

3.3.2 Image transformation

All images were resized to 224×224 pixels, since a model can only accept input images of a single common size.

3.3.3 Data augmentation

We applied data augmentation by flipping the images both vertically and horizontally, rotating them, and zooming, configured as (1) RandomFlip, (2) RandomRotation, and (3) RandomZoom. Appendix B reviews the technique in more detail.

3.4 Model construction

We used the VGG16, InceptionV3, ResNet50, and Xception architectures under two training schemes: (1) *feature extraction*, which takes the original structure and its pre-trained weights as they are; and (2) *fine-tuning* of the weights and parameters in the model, freezing 50% of the model’s weights. A classification layer was appended as the final stage to adapt each structure to Thai food classification; all four architectures share the same classification part.

The functions and hyperparameter values of the convolutional neural network models were set as follows:

- **ModelCheckpoint**, configured to save the weights that give the best accuracy.
- **ReduceLROnPlateau**, which lowers the learning rate when the model stops improving, with the minimum learning rate (`min_lr`) set to 0.001.
- **EarlyStopping** [17], which halts training when the model’s accuracy no longer improves. The `patience` parameter was set to 5 — if accuracy has not improved within the next five

epochs, training stops — with `restore_best_weights=True` so that the best weights are restored.

- **Lambda**, an anonymous function used to call a function inside a given layer of the model.
- **Optimizer**: Adam [33].
- **Activation functions**: ReLU and Softmax (defined in Appendix C).
- **Dropout** [17, 32], which randomly removes some nodes from the model during training and thereby reduces overfitting; set to 0.2.
- **Learning rate**: 0.001 for feature extraction and 0.00001 for fine-tuning.
- **Batch size**: 64.
- **Epochs**: 30.

We used four pre-trained models for Thai food classification — (1) VGG16, (2) ResNet50, (3) InceptionV3, and (4) Xception — and designed the classification layer to suit the task, with the same classification part in all four architectures. Figure 17 shows the structure of each model.

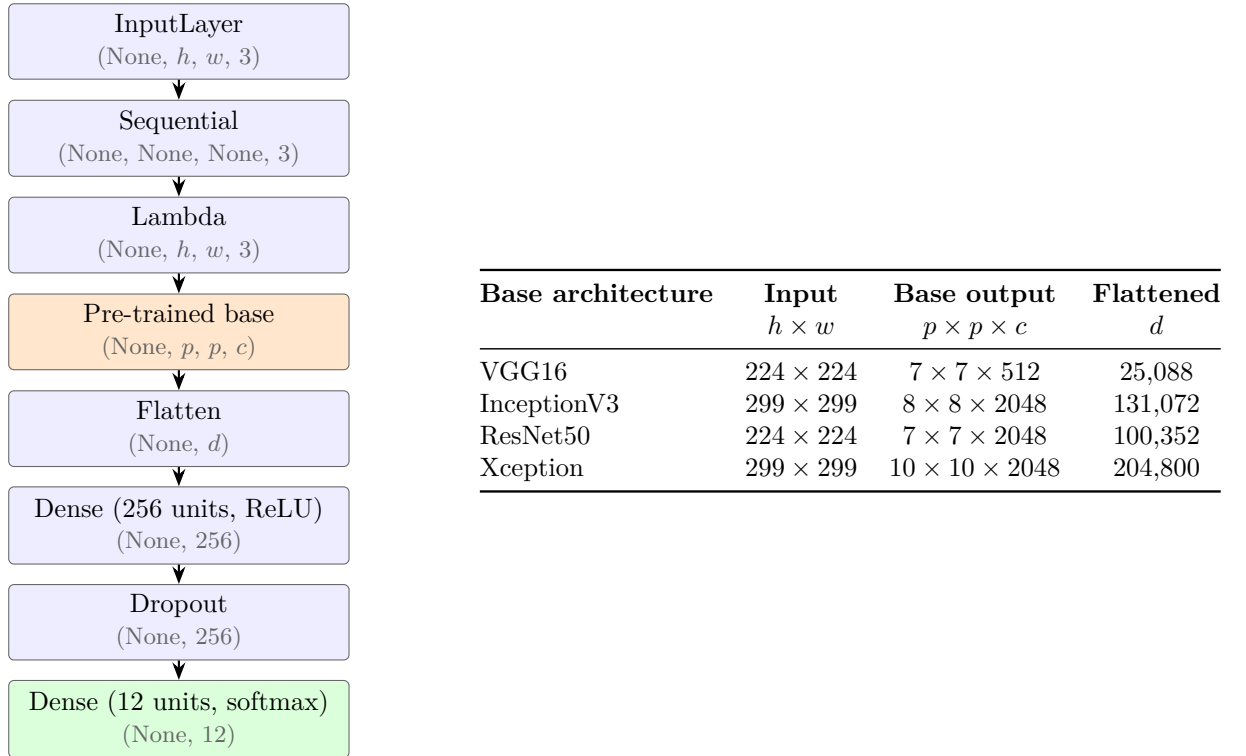


Figure 17. Structure of the classification models. All four models share the same pipeline (left): a Sequential layer and a Lambda layer feed the pre-trained convolutional base (orange), whose output is flattened and passed through a shared classification head of Dense (ReLU, 256 units), Dropout, and a 12-unit softmax output (green). The table (right) gives the input size $h \times w$, the base output shape $p \times p \times c$, and the flattened feature dimension d for each base architecture. Layer names and output shapes follow Keras conventions.

4 Results

We evaluated the performance of the convolutional neural network models built by transfer learning under the two training schemes: (1) feature extraction and (2) fine-tuning of the weights and parameters in the model by freezing weights. Performance is measured with the accuracy, precision, recall, and F1-score metrics defined in Appendix A.

Table 5. Performance of the convolutional neural network models built by transfer learning with feature extraction.

Architecture	Feature Extraction			
	Accuracy	Precision	Recall	F1-Score
Xception	86.39%	86.59%	86.39%	86.05%
ResNet50	86.11%	86.80%	86.11%	85.65%
InceptionV3	71.67%	71.59%	71.67%	70.12%
VGG16	82.50%	84.71%	82.50%	82.48%

Training the Thai food classification models by feature extraction gave the results in Table 5.

Table 5 shows that, overall, Xception is the best model, with an accuracy of 86.39%, precision of 86.59%, recall of 86.39%, and F1-score of 86.05%.

As Figure 18 shows, the accuracy and loss curves of all four models keep improving: accuracy rises and loss falls as the number of epochs grows. The accuracy curves of Xception and ResNet fluctuate considerably in the final epochs; likewise, the train-set and validation-set accuracies of InceptionV3 and VGG16 sit rather far apart, most of all for InceptionV3.

For these reasons, we improved the models by freezing weights (*fine-tuning*) so that they perform better and fit the training data more closely, letting the weights of half the model (50% of the whole) be retrained. The results are shown in Table 6.

Table 6. Performance of the convolutional neural network models built by transfer learning with fine-tuning.

Architecture	Fine Tuning			
	Accuracy	Precision	Recall	F1-Score
Xception	93.06%	93.22%	93.06%	93.02%
ResNet50	90.83%	91%	90.83%	90.72%
InceptionV3	88.89%	89.40%	88.89%	89.03%
VGG16	86.11%	87.22%	86.11%	86.17%

Table 6 shows that, overall, Xception is again the best model, with an accuracy of 93.06%, precision of 93.22%, recall of 93.06%, and F1-score of 93.02%.

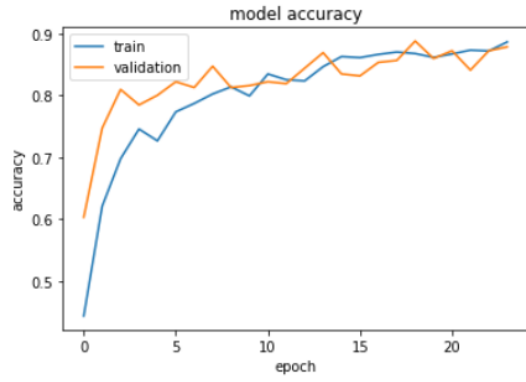
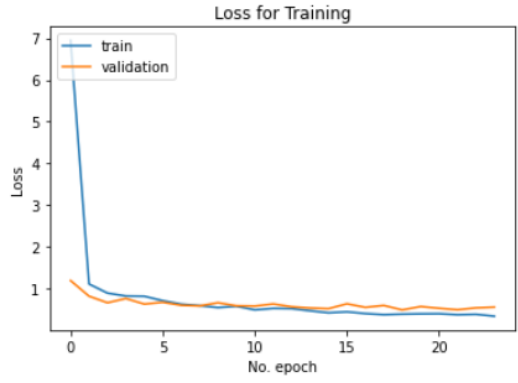
Figure 19 shows that both the accuracy and the loss curves improve as the epochs increase, but the loss curves of all four fine-tuned models trend worse than before fine-tuning. The cause may be the search for suitable weights over the retrained half — 50% of an architecture that had been pre-trained on a different dataset. By contrast, the accuracy curves of Xception and ResNet fluctuate far less in the final epochs than they did before fine-tuning.

We then evaluated per-category performance under both training schemes — feature extraction and fine-tuning — as reported in Tables 7 and 8.

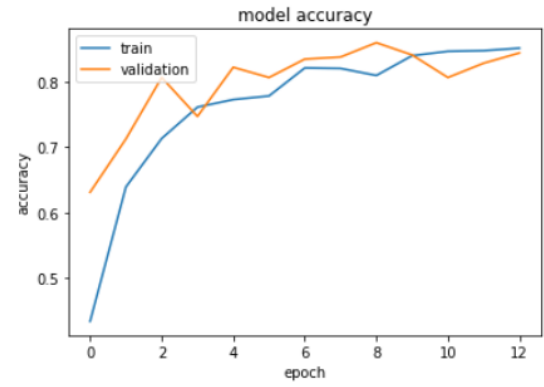
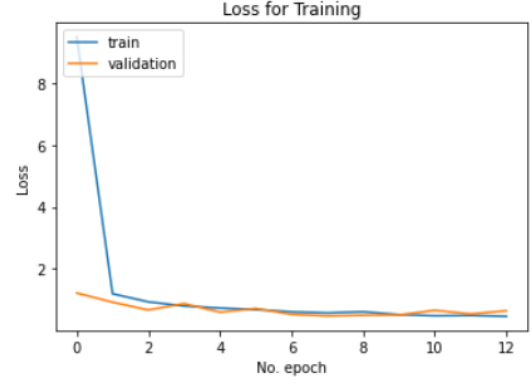
Table 8 shows that, overall, the fine-tuned Xception model predicts effectively: Pad Kra Pao, Mango Sticky Rice, Mussaman Curry, and Rice mixed with Shrimp Paste reach F1-scores of roughly 100%, 98%, 97%, and 95% respectively, the four best-performing dishes among the 12 categories. Compared with Table 7, the feature-extraction counterpart, the F1-scores of most of these four dishes improved.

Finally, we tested both Xception models on assorted images, taken from the Internet and photographed with a camera, with the results shown in Figure 20.

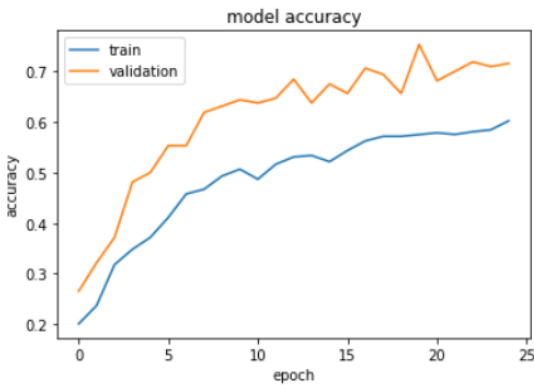
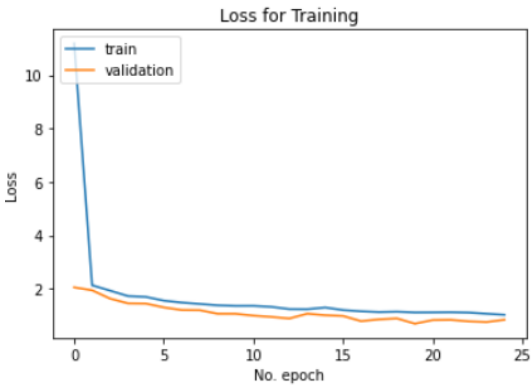
These images show that the fine-tuned Xception model predicts more accurately than its feature-extraction counterpart. Across the four test images — Mango Sticky Rice, Pad Kra Pao, Rice mixed with Shrimp Paste, and Mussaman Curry — the fine-tuned model is the more accurate of the two. The clearest case is Pad Kra Pao, which the fine-tuned model predicts correctly with fairly high probability while the feature-extraction model predicts it wrongly.



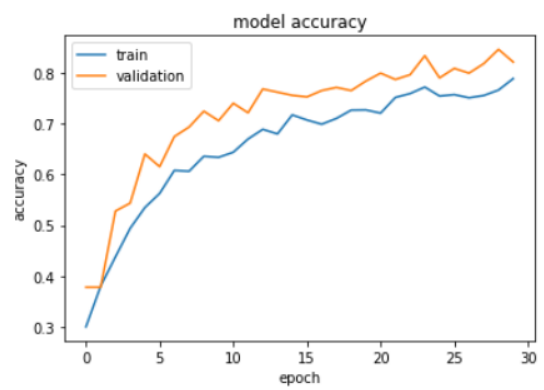
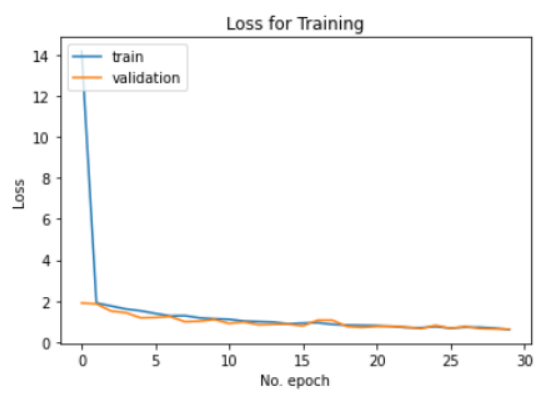
(a) Xception



(b) ResNet50

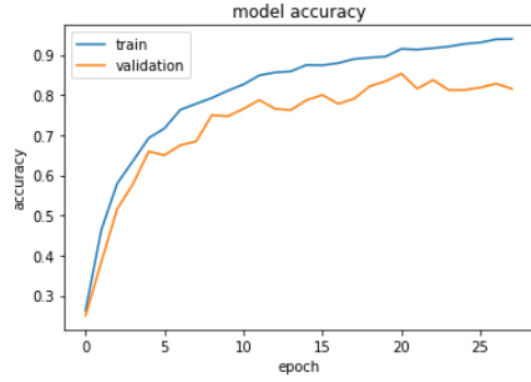
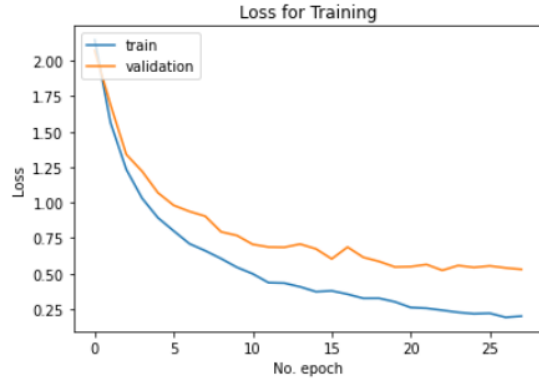


(c) InceptionV3

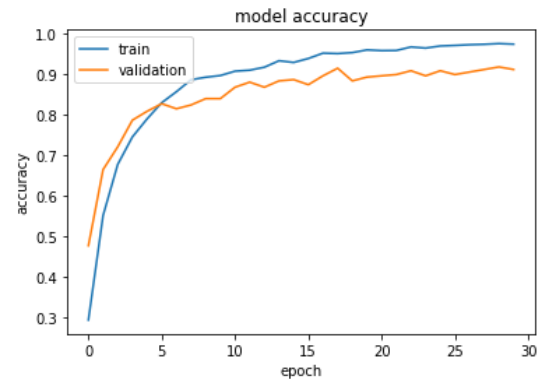
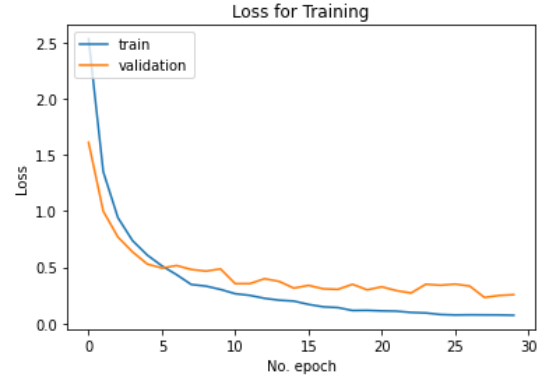


(d) VGG16

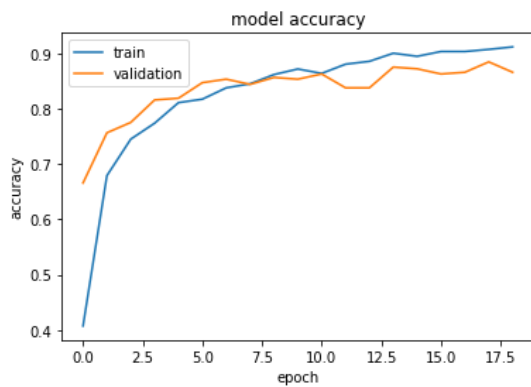
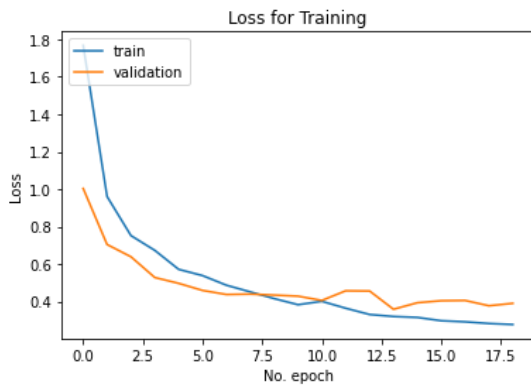
Figure 18. Training and validation loss (top) and accuracy (bottom) of the four models trained by feature extraction.



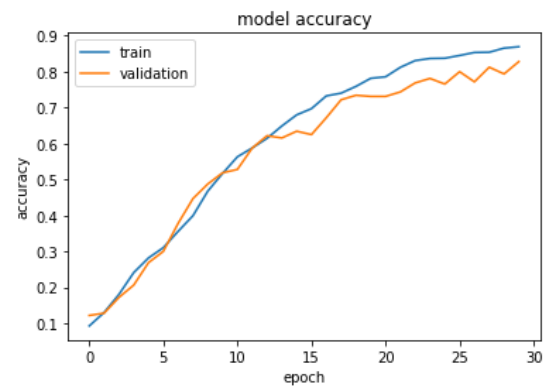
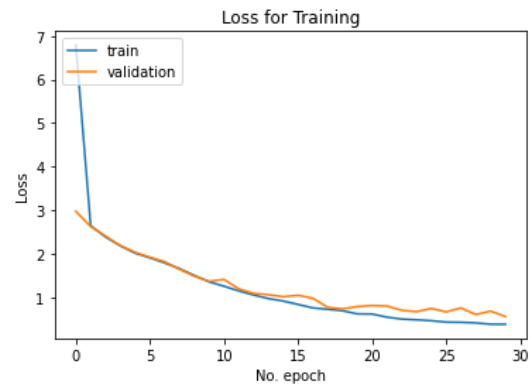
(a) Xception



(b) ResNet50



(c) InceptionV3



(d) VGG16

Figure 19. Training and validation loss (top) and accuracy (bottom) of the four models after fine-tuning.

Table 7. Accuracy, precision, recall, and F1-score on the test set for the Xception model trained by feature extraction, by food category.

Dish	Precision	Recall	F1-Score
Green Curry	76.92%	100%	86.96%
Green Papaya Salad	85.71%	100%	92.31%
Khao Soi	86.67%	86.67%	86.67%
Mango Sticky Rice	100%	96.67%	98.31%
Mussaman Curry	96.43%	90%	93.10%
Pad Kra Pao	96.77%	100%	98.36%
Pad Thai	92.31%	80%	85.71%
Rice mixed with Shrimp Paste	100%	96.67%	98.31%
Sour Soup	69.57%	53.33%	60.38%
Spicy mixed vegetable soup	83.33%	83.33%	83.33%
Tepho Curry	72.22%	86.67%	78.89%
Tom Yum Kung	79.17%	63.33%	70.37%
Accuracy	86.39%		

Table 8. Accuracy, precision, recall, and F1-score on the test set for the fine-tuned Xception model, by food category.

Dish	Precision	Recall	F1-Score
Green Curry	87.88%	96.67%	92.06%
Green Papaya Salad	93.33%	93.33%	93.33%
Khao Soi	100%	90%	94.74%
Mango Sticky Rice	100%	96.67%	98.31%
Mussaman Curry	93.75%	100%	96.77%
Pad Kra Pao	100%	100%	100%
Pad Thai	93.10%	90%	91.53%
Rice mixed with Shrimp Paste	90.91%	100%	95.24%
Sour Soup	88.89%	80%	84.21%
Spicy mixed vegetable soup	92.86%	86.67%	89.66%
Tepho Curry	93.10%	90%	91.53%
Tom Yum Kung	84.85%	93.33%	88.89%
Accuracy	93.06%		

Both Xception models misclassify Mussaman Curry, plausibly because the dishes look so much alike.



Figure 20. Predictions of the Xception model on images from the Internet and from a camera, before (a) and after (b) fine-tuning.

5 Conclusions and Discussion

We studied and presented convolutional neural network models for classifying Thai food by transfer learning, with four models — Xception, InceptionV3, ResNet50, and VGG16, all popular architectures for image classification — trained in two different ways: (1) feature extraction and (2) fine-tuning of the weights and parameters in the model. In both schemes we added techniques to obtain the best possible model, such as max pooling, dropout, early stopping, and data augmentation, and we then evaluated the models with the accuracy, precision, recall, and F1-score metrics.

The results show that, overall, the fine-tuned Xception model performs best, followed by the fine-tuned ResNet50. The fine-tuned Xception model attains an accuracy of 93.06%, precision of 93.22%, recall of 93.06%, and F1-score of 93.02%, and it predicts Thai food categories effectively: Pad Kra Pao, Mango Sticky Rice, Mussaman Curry, and Rice mixed with Shrimp Paste reach F1-scores of roughly 100%, 98%, 97%, and 95% respectively, the four highest-scoring dishes among the 12 categories.

Our findings are consistent with the work of Thuan Trong Nguyen and colleagues [20], in which the model built on the Xception architecture achieved the highest accuracy when set against the models in this article. In that study the authors used a support vector machine as the classifier in the classification layer, whereas here we used a neural network instead.

6 Recommendations

Building on this article, future work should try other architectures and widen the range of training regimes — for example, adopting other algorithms such as the support vector machine as the classifier, and adding training schemes such as fine-tuning only the last layer or fine-tuning the whole network — and then compare all the models to establish which training approach gives the best results. Because the fine-tuning step here consisted only of freezing weights, future work could also adjust the other parameters in the model, such as the learning rate and the dropout rate. Finally, since the architectures differ in character and may not take equally long to train, recording the training time of each model and analysing it alongside the performance

metrics would make model selection more precise and better suited to the task.

Appendix A Evaluation Metrics for Classification

The confusion matrix [13] is a table summarising the number of correct and incorrect predictions made by a classifier. It measures the performance of a model on classification problems and supports the computation of performance metrics such as accuracy, precision, recall, and F1-score. The entries of the confusion matrix are:

1. **True Positives (TP)**: the count of cases where the prediction matches reality: the model predicts positive and the outcome is indeed positive.
2. **True Negatives (TN)**: the count of cases where the prediction matches reality: the model predicts negative and the outcome is indeed negative.
3. **False Positives (FP)**: the count of cases where the prediction contradicts reality: the model predicts positive but the outcome is negative; also called a Type I error.
4. **False Negatives (FN)**: the count of cases where the prediction contradicts reality: the model predicts negative but the outcome is positive; also called a Type II error.

Table 9. The confusion matrix (after Suresh [13]).

		Predicted	
		Negative (−)	Positive (+)
Actual	Negative (−)	True Negative (TN)	False Positive (FP) Type I error
	Positive (+)	False Negative (FN) Type II error	True Positive (TP)

The entries of the confusion matrix are commonly combined into model performance metrics; in this article we used the following.

1. **Accuracy** is the ratio of correct predictions to all predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (3)$$

2. **Precision** is the number of positive cases classified correctly divided by the total number of predicted positive cases; in other words, of everything predicted positive, how much was predicted correctly:

$$Precision = \frac{TP}{TP + FP}. \quad (4)$$

3. **Recall** is the number of positive cases classified correctly divided by the total number of positive cases; in other words, of all the positive cases, how much was predicted correctly:

$$Recall = \frac{TP}{TP + FN}. \quad (5)$$

4. **F1-score** is a measure between 0 and 1 defined as the harmonic mean of precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}. \quad (6)$$

Appendix B Data Augmentation

Data augmentation [11] is a technique for enlarging the amount of data available to a model, either by adding copies of existing data or by synthesising new data from it. The technique eases the problem of the model learning the training data too well, known as overfitting. Slightly modified copies of the images are added to the dataset used by the model; techniques for augmenting images for machine-learning datasets include rotating the image, zooming it, adjusting its colours, and so on (Figure 21).

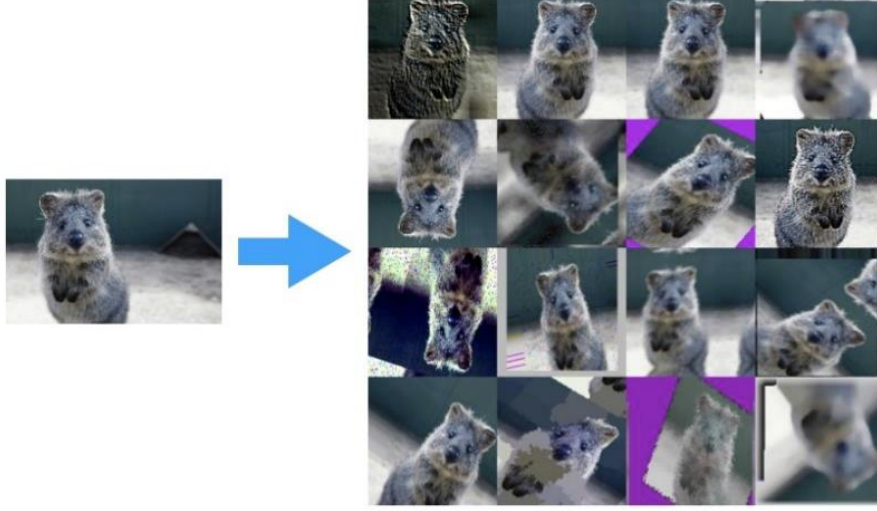


Figure 21. An original photograph of a quokka (left) with images produced from it by data augmentation (right) (source: ai.stanford.edu).

Appendix C Activation Functions

An activation function [14] takes the summed input from all input nodes and decides how much of it to pass on as output. Activation functions are inserted in both the convolutional layers and the fully connected layers. In this article we used ReLU and Softmax, described below.

Rectified Linear Unit (ReLU). ReLU [15] is used mainly in the convolutional layers to clip the output to non-negative values, and it guards against the vanishing gradient. The function is

$$f(x) = \max(0, x) = \begin{cases} 0 & \text{for } x \leq 0, \\ x & \text{for } x > 0. \end{cases} \quad (7)$$

Softmax. Softmax [16] is used in the fully connected layer to turn the final outputs into the desired classes: it takes the numerical inputs and converts them into probabilities that sum to one,

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } i = 1, \dots, K \text{ and } \mathbf{z} = (z_1, \dots, z_K) \in \mathbb{R}^K. \quad (8)$$

References

- [1] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. Online, 2017. https://www.knowledgeisle.com/wp-content/uploads/2019/12/2-Aur%C3%A9lien-G%C3%A9ron-Hands-On-Machine-Learning-with-Scikit-Learn-Keras-and-Tensorflow_-Concepts-Tools-and-Techniques-to-Build-Intelligent-Systems-0%E2%80%99Reilly-Media-2019.pdf

- [2] M. Elgendy. *Deep Learning for Vision Systems*. Online, 2020. https://library.samdu.uz/files/5c7f5c77ccc411be98a2a706bc926743_Deep_Learning_for_Vision_Systems_v6_MEAP.pdf
- [3] Dive into Deep Learning. Padding and stride. Online, n.d. https://d2l.ai/chapter_convolutional-neural-networks/padding-and-strides.html
- [4] MYO NeuralNet. Calculating the output size of convolutions and transpose convolutions. Online, 2020. <http://makeyourownneuralnetwork.blogspot.com/2020/02/calculating-output-size-of-convolutions.html>
- [5] J. J. Moolayil. A layman’s guide to deep neural networks. Online, 2019. <https://towardsdatascience.com/a-laymans-guide-to-deep-neural-networks-ddcea24847fb>
- [6] Wikipedia. Transfer learning. Online, n.d. https://en.wikipedia.org/wiki/Transfer_learning
- [7] Datagen. Understanding VGG16: concepts, architecture, and performance. Online, n.d. <https://datagen.tech/guides/computer-vision/vgg16/>
- [8] A. Sarkar. Understanding Inception: simplifying the network architecture. Online, 2020. <https://medium.com/swlh/understanding-inception-simplifying-the-network-architecture-54cd31d38949>
- [9] J. Adam. ResNet-50 API. Online, 2019. <https://jason-adam.github.io/resnet50/>
- [10] Datagen. ResNet-50: the basics and a quick tutorial. Online, n.d. <https://datagen.tech/guides/computer-vision/resnet-50/>
- [11] S. Ray. What is data augmentation? Online, 2021. <https://medium.com/lansaar/what-is-data-augmentation-3da1373e3fa1>
- [12] S. Osterburg. Xception architectural design. Online, 2020. <https://stephan-osterburg.gitbook.io/coding/coding/ml-dl/tensorflow/ch3-xception/xception-architectural-design>
- [13] A. Suresh. What is a confusion matrix? Online, 2020. <https://medium.com/analytics-vidhya/what-is-a-confusion-matrix-d1c0f8feda5>
- [14] S. Kanoktipsatharporn. What is an activation function? Online, 2019. <https://www.bualabs.com/archives/1261/what-is-activation-function-what-is-sigmoid-function-activation-function-ep-1/>
- [15] S. Kanoktipsatharporn. What is the ReLU function? Online, 2019. <https://www.bualabs.com/archives/1355/what-is-relu-function-why-popular-deep-learning-training-deep-neural-network-activation-function-ep-3/>
- [16] Wikipedia. Softmax function. Online, n.d. https://en.wikipedia.org/wiki/Softmax_function
- [17] C. Kittinaradorn. Neural network regularisation. Online, 2020. <https://guopai.github.io/ml-blog18.html>
- [18] Chayanat et al. Pushing the accuracy of Thai food image classification with transfer learning. *Engineering Journal*, 2022. <https://engj.org/index.php/ej/article/view/4465>
- [19] Vanutre. Food image classification using various CNN models. Online, 2022. https://cs230.stanford.edu/projects_fall_2019/reports/26233496.pdf
- [20] T. T. Nguyen et al. VinaFood21: a novel dataset for evaluating Vietnamese food recognition. Online, 2021. <https://arxiv.org/ftp/arxiv/papers/2108/2108.02929.pdf>
- [21] Wikipedia. Thai cuisine. Online, n.d. https://en.wikipedia.org/wiki/Thai_cuisine
- [22] Mastercard. *Global Destination Cities Index*. Mastercard, n.d.
- [23] Google Cloud. Advanced guide to Inception v3. Online, n.d. <https://cloud.google.com/tpu/docs/inception-v3-advanced>

- [24] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [25] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1409.1556.
- [26] O. Russakovsky, J. Deng, H. Su, et al. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.
- [27] C. Szegedy, W. Liu, Y. Jia, et al. Going deeper with convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015. arXiv:1409.4842.
- [28] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the Inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016. arXiv:1512.00567.
- [29] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. arXiv:1512.03385.
- [30] F. Chollet. Xception: deep learning with depthwise separable convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1251–1258, 2017. arXiv:1610.02357.
- [31] S. J. Pan and Q. Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [32] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [33] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.